

Estrutura de Dados

BFS, DFS e Conectividade

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Percorrer um Grafo: Vizinhos, Não Filhos

A árvore tinha uma raiz e um percurso natural, do topo para as folhas. O grafo não tem raiz: um ponto qualquer pode ser a origem, e o que existe são vizinhos, não filhos. Percorrer um grafo significa visitar todo ponto alcançável a partir de uma origem, sem repetir nenhum e sem se perder num ciclo, guiado apenas pela lista de vizinhos de cada ponto.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

BFS: a Fila Decide Quem Vem Primeiro

A busca em largura visita primeiro todos os vizinhos diretos da origem, depois os vizinhos desses vizinhos, e assim por camadas sucessivas. Uma fila garante essa ordem: cada ponto descoberto entra no fim da fila, e só é explorado depois de todos os pontos descobertos antes dele — exatamente o comportamento FIFO já construído em FilaDePedidos na Aula 06.



Prof. Everaldo Graciliano Souza Ribeiro

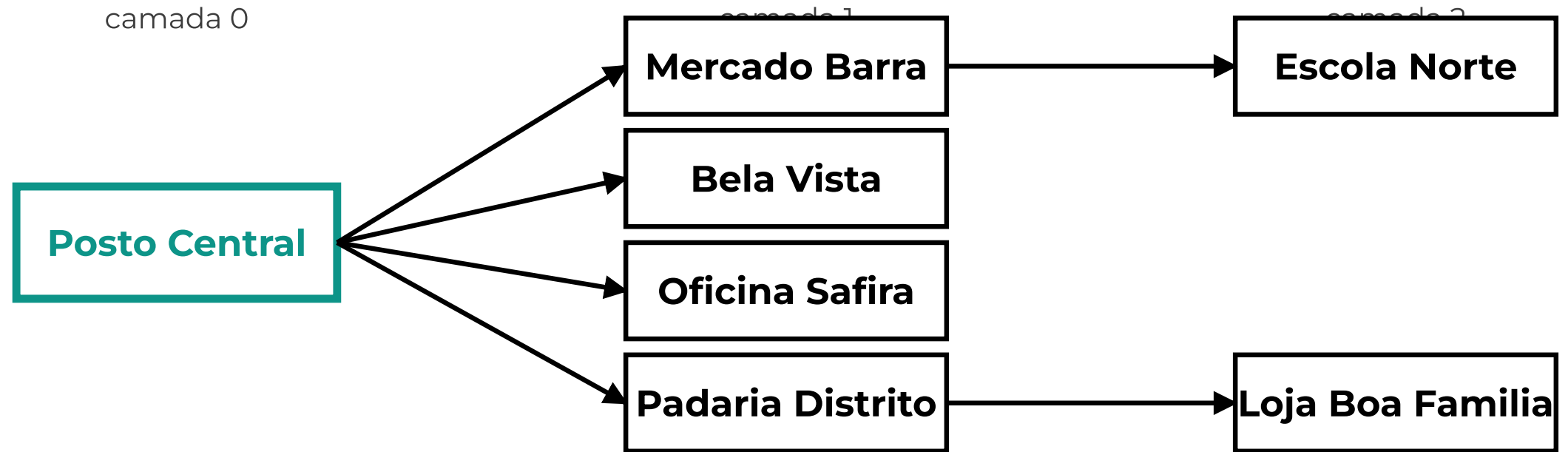
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

BFS Visita por Camadas



BFS esgota a **camada 1** inteira antes de visitar qualquer ponto da camada 2.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

bfs(): a Fila Impõe a Ordem por Camadas

```
def bfs(grafo, origem):  
    visitados = {origem}  
    ordem = []  
    fila = FilaDePedidos()  
    fila.enqueue(origem)  
    while not fila.vazia():  
        atual = fila.dequeue()  
        ordem.append(atual)  
        for vizinho in grafo.vizinhos(atual):  
            if vizinho not in visitados:  
                visitados.add(vizinho)  
                fila.enqueue(vizinho)  
    return ordem
```



DFS: a Pilha Inverte a Prioridade

A busca em profundidade mergulha por um caminho até não ter mais vizinho novo para visitar, e só então volta e tenta outro ramo. Uma pilha produz esse comportamento: o último ponto descoberto é o próximo a ser explorado, o que empurra a busca para a frente em vez de espalhá-la pelas camadas — o oposto exato do que a fila faz na BFS.



Prof. Everaldo Graciliano Souza Ribeiro

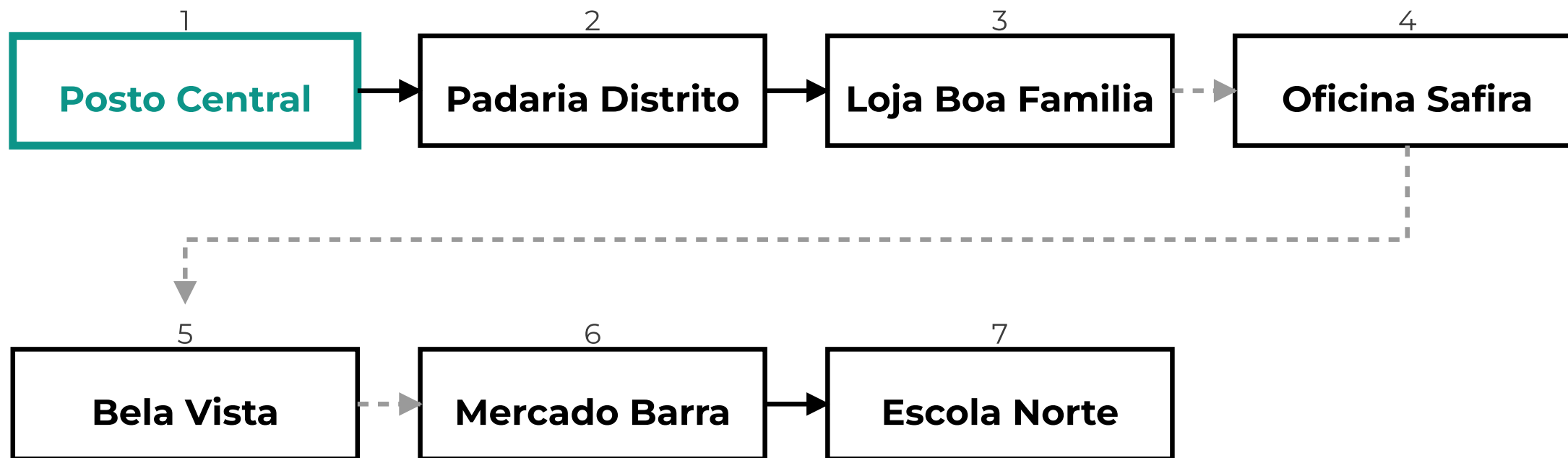
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

DFS Visita em Outra Ordem



Setas cheias seguem uma rua nova; setas tracejadas **voltam** na pilha até achar um vizinho ainda não visitado.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

dfs(): a Mesma Função, uma Linha Trocada

```
def dfs(grafo, origem):  
    visitados = {origem}  
    ordem = []  
    pilha = PilhaDeOperacoes()  
    pilha.empilhar(origem)  
    while not pilha.vazia():  
        atual = pilha.desempilhar()  
        ordem.append(atual)  
        for vizinho in grafo.vizinhos(atual):  
            if vizinho not in visitados:  
                visitados.add(vizinho)  
                pilha.empilhar(vizinho)  
    return ordem
```



Mesmo Algoritmo, Uma Estrutura Trocada

bfs() e dfs() têm exatamente a mesma forma: descobrir a origem, repetir "tira um ponto, visita, empilha ou enfileira seus vizinhos novos" até esvaziar a estrutura auxiliar. A única diferença de código é FilaDePedidos no lugar de PilhaDeOperacoes. Trocar FIFO por LIFO já basta para transformar busca em largura em busca em profundidade.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

vizinhos(): Matriz Percorre Tudo, Lista Só o Que Existe

BFS e DFS chamam `grafo.vizinhos(atual)` a cada passo, e o custo dessa chamada depende da representação escolhida na Aula 06. Em `GrafoMatriz`, descobrir os vizinhos exige varrer a linha inteira — $O(v)$, cadastrada a rua ou não. Em `GrafoLista`, o custo é $O(\text{grau do vértice})$: só o que de fato existe é percorrido, nunca as ausências.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Conectividade: a Malha Alcança a Si Mesma?

Um grafo é conexo quando existe caminho entre quaisquer dois de seus pontos. Nem toda malha cadastrada garante isso: um ponto de entrega pode existir no sistema sem que nenhuma rua o ligue ao restante da rede, tornando-o inalcançável a partir de qualquer outro endereço — um problema que nenhuma fórmula detecta sozinha, só a busca percorrendo o grafo de fato.



Prof. Everaldo Graciliano Souza Ribeiro

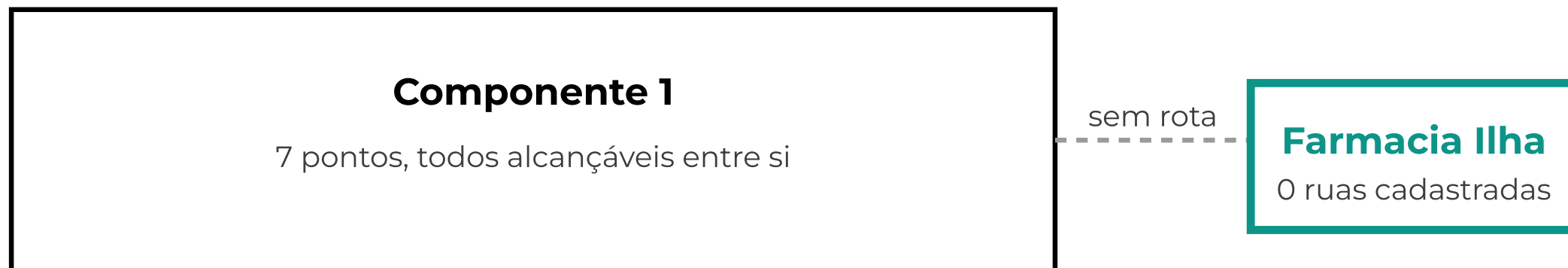
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Um Segundo Componente, Isolado



Farmacia Ilha está cadastrada, mas nenhuma rua a liga ao resto da malha — um componente à parte.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

componentes_conexos(): bfs() Reaproveitada

```
def componentes_conexos(grafo, nomes):  
    visitados = set()  
    componentes = []  
    for nome in nomes:  
        if nome in visitados:  
            continue  
        grupo = bfs(grafo, nome)  
        visitados.update(grupo)  
        componentes.append(grupo)  
    return componentes
```



Atividades

Exercício 1

Por que `bfs()` e `dfs()` visitam os mesmos 7 pontos, mas em ordens diferentes, partindo da mesma origem?

Exercício 2

Se Farmacia Ilha tivesse uma única rua ligando-a a Loja Boa Família, quantos componentes `componentes_conexos()` devolveria?

Exercício 3

Por que `grafo.vizinhos(atual)` custa $O(\text{grau do vértice})$ em `GrafoLista`, mas $O(v)$ em `GrafoMatriz`?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Escrever `dfs_recursiva(grafo, origem, visitados=None)`, sem pilha explícita, usando a pilha de chamadas da própria linguagem.
2. Criar `existe_caminho(grafo, a, b)`, reaproveitando `bfs()` para responder sim/não sem devolver a ordem inteira.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>